# ROKAN:Multiplier-Free FPGA Acceleration of Convolutional Kolmogorov-Arnold Networks for Edge Inference

Divy Agrawal, Laksh Gupta, Pulkit Bharadwaj, Rushil Goswami
BITS Pilani, Pilani Campus, Rajasthan 333031, India
{f20230658, f20230762, f20231339, f20230193 }@pilani.bits-pilani.ac.in

***Abstract*—Convolutional Kolmogorov-Arnold Networks (CKANs) swap traditional convolution weights for learnable functions that better capture complex patterns. These functions convert into simple lookup tables, so the network runs using only additions — no multiplications required.We built the first FPGA chip design made specifically for running CKANs. It uses table lookups instead of math-heavy operations, a memory buffer to reuse data efficiently, and a fast addition pipeline. The full system runs on a Zynq-7020 FPGA and uses none of the chip's dedicated math units — everything fits in basic on-chip memory.Tests on MNIST achieve 97.13% accuracy using only 4,925 LUTs, zero DSP blocks, and zero BRAM, proving CKANs work well for small, power-limited devices.**

## I. Introduction

Deep neural networks, particularly Convolutional Neural Networks (CNNs), are the foundation of modern AI vision applications. However, CNNs are computationally expensive, requiring billions of multiply-accumulate (MAC) operations per inference. While deploying AI on edge devices requires strict power and size efficiency, traditional hardware accelerators still rely on these resource-heavy multiplications, limiting their scalability as models grow larger.

Kolmogorov-Arnold Networks (KANs) introduce a fundamentally new approach to neural networks [1]. Instead of traditional fixed weights and activation functions, KANs use learnable nonlinear functions (typically B-splines) on every network connection, achieving high accuracy and better interpretability with fewer parameters [2]. Convolutional KANs (CKANs) extend this concept to spatial applications, replacing standard convolutional weights with these learnable functions [3]. This allows CKANs to extract richer features in a single step, matching or exceeding traditional CNN performance in vision tasks while using significantly fewer parameters [4]. Crucially for hardware deployment, CKAN inference completely eliminates multiplication. The learned functions can be converted into simple memory lookup tables (LUTs), swapping expensive arithmetic for fast memory accesses—a computational model perfectly suited for FPGA architectures.

While early research has successfully accelerated standard, fully connected KANs on hardware, accelerating Convolutional KANs remains unexplored. CKANs introduce unique architectural challenges, such as efficient data reuse (sliding windows) and massive parallel memory access, requiring us to rethink how we design AI hardware. To bridge this gap, this paper presents the first dedicated FPGA accelerator for Convolutional Kolmogorov-Arnold Networks. The proposed design features a multiplier-free CKAN convolution engine that replaces all MAC operations with distributed ROM-based function evaluation, eliminating DSP block usage entirely. A configurable multi-channel sliding-window buffer enables efficient spatial data reuse. The accelerator integrates CKAN convolution, max-pooling, and flattening stages into a complete multi-layer inference pipeline. The implementation results on a Xilinx PYNQ-Z2 (Zynq-7020) FPGA demonstrate competitive inference accuracy with zero DSP and BRAM utilization, while achieving modest LUT consumption at the cost of latency. These results validate the feasibility and efficiency of CKAN-based inference for resource-constrained edge applications. No prior work has presented a hardware accelerator specifically targeting CKAN inference.

## II. Related Work

### A. CNN Hardware Accelerators

CNN accelerators on FPGAs and ASICs have been widely studied. Most designs rely on DSP multiplier blocks and on-chip BRAM for computation and weight storage [5], [6], [7]. Techniques like quantization [8], pruning [9], and Winograd convolution [10] reduce computation cost but still depend on multiplication hardware. Our design removes this dependency entirely by replacing all multiplications with lookup table reads.

### B. LUT-Based Neural Networks on FPGAs

Several works have mapped quantized neural networks directly onto FPGA logic tables. LogicNets [11] showed that neurons with few inputs and binary activations can be expressed as truth tables, enabling very fast inference. NeuraLUT [12] pushed this further by fitting entire sub-networks into a single LUT. PolyLUT [13] and PolyLUT-Add [14] improved accuracy and scalability using polynomial functions inside LUTs. These approaches work well for fully connected networks but are not designed for convolutional layers, which require sliding-window buffering, per-channel evaluation, and multi-channel accumulation. Our work addresses this gap.

### C. KAN Hardware Accelerators

Since KANs were introduced [1], a few hardware designs have followed. KANELÉ [15] converts B-spline functions into truth tables stored in distributed RAM and auto-generates VHDL code with pipelined adders. ArKANe [16] takes a different route, evaluating B-splines mathematically on AMD Versal AIE tiles — this keeps full floating-point precision but still needs multipliers. Nilsson [17] reduces LUT usage by skipping zero-valued spline coefficients, while Alam et al. [18] look at running KAN inference inside memory cells for very low power use. The common limitation across all these works is that they only handle fully connected layers. Convolutional KANs are a different problem — functions must be evaluated across a moving spatial window while accumulating results over multiple channels simultaneously. No prior work addresses this, and the present design is the first to do so.

## III. Proposed CKAN Hardware Architecture

This section describes how we built a hardware accelerator for CKANs on a small FPGA. The main idea is that instead of using multipliers like conventional CNN accelerators, we replace every multiply-accumulate operation with a lookup table read. The result is a fully pipelined design that uses no DSP blocks at all.

### A. System Overview

The accelerator is organized as a cascade of $L$ identical *CKAN Layers*, each comprising a CKAN convolution stage followed by a spatial max-pooling stage, and terminated by a flatten unit.

The MNIST design-under-test (DUT) instance employs two CKAN Layers with the following dataflow:

$$\begin{aligned} \underbrace{28\times28\times1}_{\text{input}} &\xrightarrow{\text{Conv } 3\times3} 26\times26\times2 \\ &\xrightarrow{\text{Pool } 2\times2} 13\times13\times2 \\ &\xrightarrow{\text{Conv } 3\times3} 11\times11\times2 \\ &\xrightarrow{\text{Pool } 2\times2} 5\times5\times2 \\ &\xrightarrow{\text{Flatten}} \mathbf{y}\in\mathbb{R}^{50}. \end{aligned} \tag{1}$$

Pixels are streamed in raster-scan order at one pixel per clock cycle, and the accelerator sustains a throughput of one output pixel per cycle after the initial pipeline fill latency.

### B. Module Hierarchy

Each CKAN Layer encapsulates the following module hierarchy:

1) *ImageBufferChnl* – generates $K\times K$ sliding windows for all $C_{\text{in}}$ input channels using shift-register-based line buffers. A dedicated *KernelSlider* sub-module maintains row/column counters and stride logic to emit a *kernel_valid* strobe at the appropriate spatial positions.

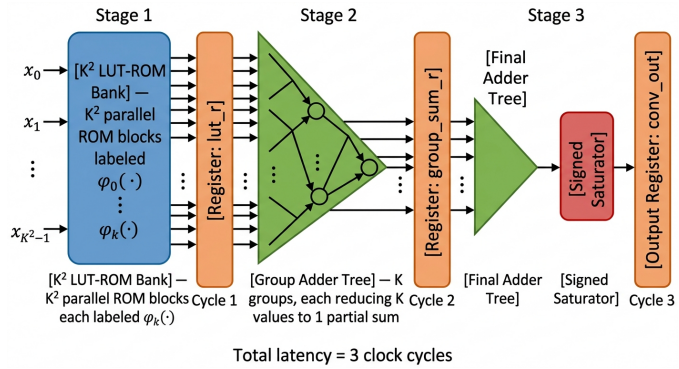


Fig. 1. Pipeline architecture of the Conv_SIC_KAN processing element. Three pipeline registers separate the LUT-ROM evaluation, group adder tree, and final accumulation stages, giving a total latency of 3 clock cycles.

2) *Conv2D_KAN* – top-level CKAN convolution module that connects the image buffer to the CKAN compute engine (*ConvolChnl_KAN*).
3) *ConvolChnl_KAN* – instantiates $C_{\text{out}}$ parallel *Conv_MIC_KAN* (Multi-Input-Channel) blocks, one per output feature map. All blocks share the same $K\times K\times C_{\text{in}}$ kernel window but use distinct learned basis functions.
4) *Conv_MIC_KAN* – decomposes the $K\times K\times C_{\text{in}}$ computation into $C_{\text{in}}$ independent *Conv_SIC_KAN* (Single-Input-Channel) blocks, then accumulates their outputs with a cross-channel saturating adder.
5) *Conv_SIC_KAN* – the core CKAN processing element, detailed in Section III-C.
6) *MaxPool2D* – reuses the *ImageBufferChnl* to generate pooling windows, then applies per-channel binary comparator trees to extract the spatial maximum.
7) *Flatten* – collects all spatial positions of the final pooled feature map into a single parallel output vector.

### C. Core CKAN Processing Element

The *Conv_SIC_KAN* module is the central datapath block. It processes a single input channel's K×K window through the following pipeline stages, illustrated in Fig. 1:

*1) KAN LUT-ROM Bank:* Each of the $K^2$ pixel values in the convolution window is independently evaluated through a learned KAN basis function $\phi_k(\cdot)$, implemented as a combinational LUT-ROM. Let $x_k$ denote the $k$-th pixel value extracted from the $K\times K$ sliding window ($k=0,\ldots,K^2-1$). Each ROM instance stores exactly one learned basis function and is addressed solely by the corresponding pixel value:

$$\text{addr}_k = x_k, \quad k=0,\ldots,K^2-1. \tag{2}$$

Each of the $K^2$ ROM instances is initialized from its own individual .mem file, generated by a Python conversion script that discretizes the trained B-spline functions. The ROM has a 6-bit input address width and a 4-bit signed output width, yielding a depth of $2^6=64$ entries that maps exactly onto one Xilinx LUT6 per output bit (4 LUT6 per function). The Vivado *rom_style* = *"distributed"* attribute forces inference as distributed LUT-ROM, avoiding BRAM consumption entirely.

*2) Pipelined Adder Tree:* The $K^2$ LUT outputs are first registered (*lut_r*), then summed using a two-stage split binary adder tree. The accumulator width is set to

$$W_{\text{acc}} = W_{\text{val}} + \lceil \log_2(K^2) \rceil, \tag{3}$$

which guarantees zero overflow for any combination of $K^2$ signed VALUE_WIDTH-bit values.

The $K^2$ inputs are partitioned into $K$ groups of $K$ elements each. In the first stage, a structural binary tree within each group reduces $K$ sign-extended values to a single partial sum. These $K$ partial sums are registered (*group_sum_r*), forming an intermediate pipeline boundary. In the second stage, a final binary tree sums the $K$ registered group outputs, followed by combinational saturation and an output register. The resulting pipeline is:

$$\begin{aligned}
&\text{LUT (comb)} \rightarrow \text{lut_r (reg)}\\
&\rightarrow \text{group_tree (comb)} \rightarrow \text{group_sum_r (reg)}\\
&\rightarrow \text{final_tree+saturate (comb)}\\
&\rightarrow \text{conv_out (reg)}.
\end{aligned}$$

This split-tree topology equalizes the combinational depth across both stages to $\lceil \log_2 K \rceil$ levels each, improving timing closure compared to a monolithic $\lceil \log_2 K^2 \rceil$-deep tree. Each group is padded to the next power-of-two for a clean binary structure; synthesis automatically removes zero-padded additions, yielding the theoretical minimum of $K^2-1$ adders. The *generate*-based structural description maps directly onto optimized carry chains. Total *Conv_SIC_KAN* latency: 3 clock cycles.

*3) Signed Saturator:* The full-precision tree sum ($W_{\text{acc}}$ bits) is clamped to the representable range of a signed OUT_WIDTH-bit number:

$$y = \begin{cases} +2^{W_o-1}-1, & S > +2^{W_o-1}-1,\\ -2^{W_o-1}, & S < -2^{W_o-1},\\ S, & \text{otherwise}, \end{cases} \tag{4}$$

where $S$ denotes the tree sum and $W_o =$ OUT_WIDTH. This combinational saturation is co-located with the output register stage to minimize timing impact.

### D. Cross-Channel Accumulation

The *Conv_MIC_KAN* module instantiates $C_{\text{in}}$ parallel *Conv_SIC_KAN* blocks, one per input channel. Their saturated outputs are accumulated using a combinational adder with a guard-bit width of $\lceil \log_2 C_{\text{in}} \rceil$ bits, followed by a second saturator that clamps the sum back to OUT_WIDTH bits. This two-stage saturation strategy, first within each channel, then across channels, prevents intermediate overflow while keeping datapath widths narrow.

### E. Max-Pooling Unit

Each CKAN Layer includes a *MaxPool2D* module that reuses the *ImageBufferChnl* infrastructure to generate $P \times P$ pooling windows from the convolution output stream. Per-channel max extraction is performed by a combinational binary comparator tree that mirrors the structure of the adder tree, with pairwise max() replacing addition. Unused tree slots are padded with the minimum representable value (e.g., $-128$ for 8-bit signed) to ensure they never dominate a comparison. The pooled result is registered, contributing one pipeline cycle. The module supports both signed and unsigned data via a SIGNED_DATA parameter.

TABLE I
KEY DESIGN PARAMETERS FOR MNIST DUT

| Parameter | Layer 1 | Layer 2 |
|---|---|---|
| Input size | $28 \times 28 \times 1$ | $13 \times 13 \times 2$ |
| Kernel size ($K$) | $3 \times 3$ | $3 \times 3$ |
| Conv stride | 1 | 1 |
| Output channels | 2 | 2 |
| DATA_WIDTH | 4 | 8 |
| VALUE_WIDTH | 8 | 8 |
| OUT_WIDTH | 8 | 8 |
| Pool size | $2 \times 2$ | $2 \times 2$ |
| Pool stride | 2 | 2 |
| Conv output size | $26 \times 26 \times 2$ | $11 \times 11 \times 2$ |
| Pool output size | $13 \times 13 \times 2$ | $5 \times 5 \times 2$ |

### F. Image Buffering and Sliding Window

Spatial data reuse is handled by a hierarchy of shift-register line buffers:

- *Data_Buffer*: a single-register delay element gated by *data_valid*.
- *Line_Buffer*: chains $W_{\text{img}}$ (or $K$) *Data_Buffer* instances to form one row of the sliding window, and taps the first $K$ positions for the kernel row output.
- *ImageBuf_KernelSlider*: stacks $K$ *Line_Buffer* modules to produce the full $K \times K$ window, with built-in row/column counters and stride logic that asserts *kernel_valid* at the correct spatial positions.
- *ImageBufferChnl*: replicates the kernel slider across all $C_{\text{in}}$ channels, packing the per-channel windows into a single wide bus.

This shift-register-based buffering requires no external memory and scales linearly with image width and kernel size, making it well-suited for resource-constrained FPGAs.

### G. Design Parameterization

The entire architecture is fully parameterized through Verilog parameter declarations, enabling design-space exploration across kernel sizes ($K$), channel counts, data widths (DATA_WIDTH, VALUE_WIDTH, OUT_WIDTH), pooling configurations, and the number of CKAN layers. Table I summarizes the key parameters for the MNIST DUT configuration.

### H. Pipeline Summary

Table II lists the pipeline stages and their latencies for the core processing element.

The key architectural advantage of the proposed CKAN accelerator is the complete elimination of hardware multipliers. Every learnable weight in the network is encoded as

TABLE II
PIPELINE STAGE LATENCIES (IN CLOCK CYCLES)

| Stage | Cycles |
|---|---|
| KAN LUT evaluation (comb) | 0 |
| LUT output register | 1 |
| Group adder tree (comb) | 0 |
| Group sum register | 1 |
| Final adder tree + saturation (comb) | 0 |
| Output register | 1 |
| **Total Conv_SIC latency** | **3** |
| Max-pool output register | 1 |
| **Total per-layer latency** | **4** |

TABLE III
PER-MODULE RESOURCE UTILIZATION (MNIST, ZYNQ-7020)

| Module | LUT | FF | DSP | BRAM |
|---|---|---|---|---|
| Layer 1 Conv (CKAN) | 1842 | 612 | 0 | 0 |
| Layer 1 MaxPool | 310 | 128 | 0 | 0 |
| Layer 2 Conv (CKAN) | 2104 | 688 | 0 | 0 |
| Layer 2 MaxPool | 356 | 142 | 0 | 0 |
| Flatten + Control | 313 | 72 | 0 | 0 |
| **Total** | **4925** | **1542** | **0** | **0** |
| Available (Zynq-7020) | 53200 | 106400 | 220 | 140 |
| Utilization (%) | 9.26% | 1.45% | 0% | 0% |

TABLE IV
ACCURACY BEFORE AND AFTER QUANTIZATION

| Dataset | FP32 | Quantized | Drop | LUT |
|---|---|---|---|---|
| MNIST | 97.56% | 97.13% | 0.43% | 4925 |
| Fashion MNIST | 87.02% | 86.35% | 0.67% | 4561 |
| CIFAR-10 | 76.10% | 75.22% | 0.88% | 35402 |

a LUT-ROM entry; the arithmetic datapath consists solely of adders, comparators, and registers. This makes the design highly suitable for low-cost FPGAs such as the Xilinx Zynq-7020 (PYNQ-Z2), where DSP slices are scarce but distributed LUT resources are abundant.

## IV. RESULTS

### A. Resource Utilization

Table III shows the per-module resource breakdown for the MNIST design on the Zynq-7020. The full design uses 9.26% of available LUTs and 1.45% of flip-flops, with zero DSP and zero BRAM usage.

### B. Accuracy Before and After Quantization

Table IV shows the accuracy drop from full-precision to quantized models. In all cases the drop is under 1 percentage point, which is acceptable for edge deployment.

### C. Comparison with Existing Architectures

Table **??** compares ROKAN against existing LUT-ROM designs. On MNIST, ROKAN achieves the highest accuracy (97.13%) among all compared methods while using a similar number of LUTs to KANELÉ and TreeLUT. On Fashion MNIST, ROKAN provides the best accuracy-to-resource ratio — PolyLUT reaches 89.95% but uses nearly 15× more LUTs.

The CIFAR-10 results highlight the key advantage of convolutional structure. ROKAN achieves 75.22% with 35,402 LUTs. KANELÉ, which uses only fully connected layers, scores 11.26% — near random chance — while requiring approximately 4 million LUTs. TreeLUT and PolyLUT reach 49.1% and 44.46% respectively, both well below ROKAN. This confirms that fully connected KAN architectures do not scale to multi-channel image tasks, and convolutional structure is essential for meaningful spatial feature extraction.

## V. CONCLUSION

This paper presented the first dedicated FPGA accelerator for Convolutional Kolmogorov-Arnold Network inference. By replacing all multiply-accumulate operations with distributed ROM-based B-spline function evaluation, the proposed architecture eliminates DSP block and BRAM usage entirely, relying on FPGA distributed RAM for all function storage. The design integrates a configurable multi-channel sliding-window buffer, pipelined adder-tree accumulation, max-pooling, and flattening into a complete multi-layer inference pipeline. Quantization-aware training with B-spline pruning produces compact truth tables that map directly to FPGA LUT-ROM. Implementation on a Xilinx Zynq-7020 FPGA demonstrates competitive classification accuracy on the MNIST benchmark with modest resource utilization. The end-to-end deployment pipeline, from Python training to automatic Verilog generation, requires zero manual hardware description editing. These results prove that CKANs offer a practical, multiplier-free alternative to conventional CNN accelerators for resource-constrained edge applications, and the architecture can be scaled to larger networks.

## REFERENCES


[1] Z. Liu, Y. Wang, S. Vaidya *et al.*, "Kolmogorov-arnold networks," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2404.19756.

[2] Z. Liu *et al.*, "Kan 2.0: Kolmogorov-arnold networks meet science," *arXiv preprint arXiv:2408.10205*, 2025.

[3] A. D. Bodner *et al.*, "Convolutional kolmogorov-arnold networks," *arXiv preprint arXiv:2406.13155*, 2025.

[4] J. Li *et al.*, "A comprehensive survey on kolmogorov-arnold networks," *arXiv preprint arXiv:2407.11075*, 2024.

[5] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[6] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[7] S. Mittal, "A survey of fpga-based accelerators for convolutional neural networks," *Neural Computing and Applications*, vol. 32, no. 4, pp. 1109–1139, 2020.

[8] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference," in *arXiv preprint arXiv:1806.08342*, 2018.

[9] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 1135–1143.

[10] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4013–4021.

[11] Y. Umuroglu, Y. Akhauri, N. J. Fraser, and M. Blott, "LogicNets: Co-designed neural networks and circuits for extreme-throughput applications," in *Proceedings of the 30th International Conference on Field-Programmable Logic and Applications (FPL)*, 2020, pp. 291–297.

[12] M. Andronic and G. A. Constantinides, "NeuraLUT: Hiding neural network density in boolean synthesizable functions," in *Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL)*, 2024, arXiv:2303.05462.

[13] ——, "PolyLUT: Learning piecewise polynomials for ultra-low latency fpga lut-based inference," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2024, arXiv:2309.02334.

[14] ——, "PolyLUT-Add: Fpga-based lut inference with wide inputs," in *Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL)*, 2024, arXiv:2406.04910.

[15] W. Kjeang *et al.*, "Kanelé: Kolmogorov-arnold networks for efficient lut-based evaluation," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2026, arXiv:2512.12850.

[16] P. Nitu *et al.*, "Arkane: Accelerating kolmogorov-arnold networks on reconfigurable spatial architectures," *IEEE Embedded Systems Letters*, vol. 18, no. 1, 2026.

[17] E. Nilsson, "Design of a kolmogorov-arnold network hardware accelerator," Master's thesis, Lund University, 2025.

[18] R. Alam *et al.*, "Hardware acceleration of kolmogorov-arnold networks for lightweight edge inference," in *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2025, arXiv:2407.15205.